

Graph-Conditioned Neural Decompilation of Dart AOT Binaries: Pass@k, Reward Adaptation, and Multi-Arm Candidate Ensembling

Raafat Abualazm

Ayman AboElHassan

Amr G. Wassal

Abstract

Neural decompilation is increasingly framed as code generation, but surface similarity and syntactic validity can diverge sharply from functional correctness. Prior Dart AOT work showed that small fine-tuned language models can generate idiomatic Dart and that CodeBLEU and compile@k capture useful but incomplete aspects of decompilation quality. A later HumanEval-Dart study found that conventional fine-tuning can improve CodeBLEU and compile@k while failing to improve pass@k. This paper asks whether explicit binary structure and inference-time candidate selection recover functional behavior that sequence-only fine-tuning misses.

We introduce a graph-conditioned decompilation architecture that encodes control-flow and data-flow structure from x86-64 Dart AOT assembly with a GraphCodeBERT-based encoder, then conditions a Qwen3-8B-Base decoder through learned graph prefix tokens. We evaluate architecture ablations, reward-tuned GRPO variants, rejection-sampling supervised fine-tuning, deployable compile-aware reranking, and multi-arm candidate ensembling on 154 HumanEval-Dart tasks, and we situate all of them against strong general-purpose frontier models prompted zero-shot on the same assembly. The frontier comparison sets the central reference point: a leading frontier model (GPT-5.5) recovers functionally correct Dart far more often than any specialized arm, reaching pass@10 of 0.701 versus 0.182 for the best fine-tuned arm and 0.279 for a 17-arm union; three further frontier models also exceed the union. An independent GPT-5.5 prompt ablation reaches pass@10 of 0.714 with full assembly and 0.701 under an approximate 2,048-assembly-token cap (110 versus 108 covered tasks; exact McNemar $p = 0.727$), indicating that context length alone does not explain the frontier gap. In a matched cleaned-assembly v2 comparison, assembly alone covers 109 tasks, while adding an address-aligned CFG or organizing all instructions by CFG covers 108; neither paired difference is significant. For HumanEval-scale Dart AOT functions, general frontier capability currently dominates specialized 8B training. Within the specialized regime – the setting that matters when binaries cannot be sent to a hosted API – graph conditioning yields the strongest CodeBLEU and competitive functional scores, a binary-reward GRPO arm gives the highest specialized pass@10 (0.182), and no single arm dominates on functional metrics. Errors are complementary: pooling 17 adapted arms yields at least one passing candidate for 43 of 154 tasks, compared with 24 for the clean graph-conditioned arm and 28 for the best GRPO arm. We explicitly treat RS-SFT and pooled-arm results as benchmark candidate-coverage analyses, not held-out generalization claims. Overall, the results argue that the near-term value of specialized small Dart AOT decompilers lies in deployability, cost, and distillation from frontier teachers rather than in out-recovering them, and that multi-policy candidate pooling exposes complementary recoverable cases that future deployable selection mechanisms should exploit.

Index Terms—Neural decompilation, reverse engineering, Dart AOT, graph-conditioned code models, pass@k, CodeBLEU, reinforcement learning, reranking.

1 Introduction

Decompilation reconstructs high-level source-like programs from low-level binaries. For reverse engineering, malware analysis, security auditing, and software maintenance, the useful output is not merely compilable pseudocode. It should be readable, idiomatic, and functionally faithful to the original program. Traditional decompilers recover valuable control and data-flow information, but their outputs often contain generic identifiers, low-level control structures, and C-like artifacts that are far from the original source language.

Neural decompilation treats this recovery problem as translation or code generation. Katz et al. showed that sequence models can learn some assembly-to-source patterns, but their RNN setting predates modern code LLMs and focuses on small C-like snippets [14]. DIRE and DIRTY then shifted attention to a key missing artifact in decompiler output: meaningful variable names and types [15, 6]. SLaDe and LLM4Decompile moved closer to end-to-end neural decompilation, showing that Transformer and LLM systems can improve readability and re-executability for C-family binaries [3, 22]. More recent systems such as CodeInverter and Nova show that structure-aware LLM decompilation is already an active direction [17, 13]. These systems are important baselines and motivation, but they do not settle the managed-language case. Dart is especially challenging: production Flutter applications are compiled Ahead-of-Time (AOT), and the resulting native code reflects type specialization, inlining, and runtime-specific object representations.

Prior Dart-focused work showed that small specialized models can generate readable Dart and can approach much larger code models on CodeBLEU under limited-data conditions [1]. The follow-up HumanEval-Dart study showed why that result is not enough: fine-tuning can improve CodeBLEU and compile@k while failing to improve, or even regressing, pass@k functional correctness [2]. This paper is the architectural and selection-oriented extension of those studies: the earlier work established the target and metric mismatch, while this work tests graph-conditioned decoding, reward adaptation, and candidate selection against pass@k. We therefore treat functional correctness as the central evaluation target and use CodeBLEU and compile@k as diagnostic metrics rather than final evidence of decompilation success.

Motivated by the finding that sequence-only fine-tuning does not reliably improve functional correctness, we evaluate whether graph-conditioned decoding, reward adaptation, reranking, and candidate pooling recover additional semantic fidelity. We investigate two complementary ideas. First, decompilation is not plain text translation: assembly has explicit control-flow and data-flow structure. We therefore condition a causal decoder on graph-derived representations built from extracted basic blocks, control-flow edges, and lightweight data-flow edges. Second, pass@k is a tail metric: it rewards the existence of at least one correct candidate among many. A single checkpoint optimized for expected per-sample reward may narrow its output distribution and reduce the diversity needed for pass@k. We therefore distinguish single-model performance from multi-arm candidate coverage and reranking. Throughout, we anchor these specialized-model results against strong general-purpose frontier models prompted zero-shot on the same assembly, so that every specialized number is read against the capability a practitioner could obtain today by calling a hosted model.

The central finding is that general-purpose frontier models, prompted zero-shot, currently dominate this task: on the 154-task pass set, GPT-5.5 reaches pass@10 of 0.701 and three further frontier models exceed the 0.279 coverage of a 17-arm specialized union, none of which received any decompilation-specific training. A separate GPT-5.5 prompt-representation ablation shows that an approximate 2,048-token assembly cap changes task coverage by only two tasks relative to its full-assembly control and that neither compact textual CFG augmentation nor CFG organization produces a statistically distinguishable gain over its matched assembly-only control. This sets an honest ceiling and reframes the specialized-model contributions as a study of the deployable regime – where proprietary or malicious binaries cannot be sent to a hosted API – rather than as a claim to beat frontier capability. Within that regime, three findings hold. Learned graph conditioning gives the strongest similarity profile and competitive functional scores while avoiding a

long raw-assembly decoder prompt; a binary-reward GRPO arm gives the highest specialized pass@10 but does not translate into a general functional win once pass-aligned compile measurement is applied; and the largest functional gains appear not in any single checkpoint but as complementary candidates spread across many imperfect arms, whose union covers 43 of 154 tasks. Current specialized models therefore sometimes place correct solutions in their sampled tails, but no single policy or reranker reliably selects them, and none matches a frontier zero-shot. We separate these claims throughout: clean single-arm comparisons support the architectural analysis, the frontier comparison supplies the capability reference, and reranking and multi-arm union results are treated as candidate-discovery and selection analyses rather than clean generalization evidence.

2 Related Work

2.1 Neural decompilation and binary-to-source recovery

Katz et al. framed neural decompilation as sequence-to-sequence recovery from binary code to source-like text [14]. Their result is important because it showed that learned models can replace some hand-engineered decompiler stages, but it also exposed the limits of flat sequence modeling. DIRE and DIRTY address a different part of the information-loss problem: they recover identifiers and types for decompiler output rather than generating a complete source program from assembly [15, 6]. ReSym pushes that direction further by combining LLMs with program analysis to recover variable and data-structure symbols from stripped binaries [25]. These works motivate learned semantic recovery, but they assume C/C++-oriented artifacts and do not model Dart AOT runtime conventions.

More recent systems attempt end-to-end assembly-to-C recovery. Beyond the C argues for retargetable neural translation rather than language-specific decompiler engineering [10]. SLDe shows that a smaller neural model can become practical when paired with type inference and evaluated across ISAs and optimization levels [3]. LLM4Decompile scales the approach with open LLMs trained over large assembly/source corpora and evaluates re-executability on HumanEval-derived and ExeBench-derived benchmarks [22]. CodeInverter augments LLM decompilation with control-flow and data-mapping information, while Nova trains an assembly-oriented language model with hierarchical structure and contrastive learning [17, 13]. These systems make clear that structural information is not, by itself, a new idea. Our distinction is narrower: Dart AOT as the target, learned graph-prefix conditioning rather than only serialized structural prompts, and a pass@k-centered study of reward adaptation, reranking, and multi-arm coverage. Decompile-Bench further expands the field with million-scale binary-source pairs for real-world C decompilation [23]. Recent neural decompilation work also studies joint code and type recovery, underscoring that source recovery and semantic annotation are coupled problems [7].

Existing Dart and Flutter reverse-engineering tools occupy a different point in the design space. Tools such as B(l)utter, reFlutter, and JEB’s Dart AOT snapshot helper focus on snapshot introspection, runtime patching, metadata recovery, or analyst assistance for Flutter applications [24, 12, 8]. They are important operational baselines for Dart reverse engineering, but they do not attempt the same source-level, test-executed neural decompilation task studied here.

2.2 Code metrics and functional correctness

CodeBLEU was designed to improve over lexical BLEU by adding syntax and data-flow components [20]. That makes it a better similarity metric for code, but it still measures agreement with a reference rather than executable behavior. HumanEval introduced pass@k as a functional code-generation metric: the question is whether at least one of k sampled programs passes tests [5]. MultiPL-E extended this style of evaluation

across many programming languages, including Dart in its polyglot benchmark family [4]. Our HumanEval-Dart setting follows the same broad idea of translated unit-test tasks, but pairs each task with Dart AOT assembly and graph artifacts for decompilation. The earlier Dart AOT metric study connected these lines and showed that CodeBLEU, compile@k, and pass@k can move in different directions on neural decompilation [2]. We therefore report CodeBLEU and compile@k, but we interpret them as supporting diagnostics. pass@k and task-level coverage are the main evidence for semantic recovery.

2.3 Candidate selection and reranking

Candidate selection has a long precedent in code generation. AlphaCode generated large pools of programs, filtered them, clustered similar solutions, and selected a small submission set for programming-contest problems [16]. Our compile-cluster reranker uses the same broad filter-and-cluster philosophy, but in a different setting: decompilation from assembly to Dart, where hidden tests cannot be used by a deployable selector. We therefore separate deployable compile-aware reranking from oracle reranking, which is reported only as an upper bound on candidate-pool quality.

2.4 Graph-conditioned code modeling

GraphCodeBERT demonstrates that data-flow structure can improve code representations over flat token sequences [9]. We use the same principle, but the direction is different. GraphCodeBERT extracts structure from source code for representation learning; our pipeline extracts CFG/DFG-like structure from assembly and uses it to condition a generative decoder. This lets the decoder receive binary structure without forcing thousands of assembly tokens into the prompt. CodeInverter is the closest methodological prior art because it also injects control-flow and data-mapping information into LLM decompilation [17]; our version tests a learned graph-encoder-to-prefix path for Dart AOT rather than a serialized structure prompt for C decompilation.

2.5 Adaptation and preference optimization

LoRA makes these experiments feasible by adapting large pretrained models with low-rank trainable updates instead of full fine-tuning [11]. We apply LoRA to both the GraphCodeBERT encoder and the Qwen3 decoder, using the exact Qwen/Qwen3-8B-Base checkpoint as the code and language prior [26]. For reward adaptation, we experiment with GRPO, which DeepSeekMath introduced as a critic-free group-relative variant of PPO [21]. Reference-free preference objectives such as SimPO show that sequence-level log-probability rewards can simplify alignment by removing the explicit reference model [18]. We also evaluate a SimKO-style pass@k-oriented grouping arm, motivated by SimKO’s analysis that RLVR can over-concentrate probability mass on top-1 candidates and suppress pass@K exploration [19]. Recent RLVR analyses similarly warn that reward optimization can improve some reasoning outcomes while changing the relationship between pass@1, pass@k, and sample diversity [27]. Our results show the corresponding danger in decompilation: pass@k rewards diversity over a sample pool, while online reward optimization can make a single policy narrower.

3 Method

Figure 1 summarizes the architecture and evaluation loop. The key design choice is to avoid treating the assembly listing as another long decoder prompt. G3 is not a topology-only model: the graph/local block

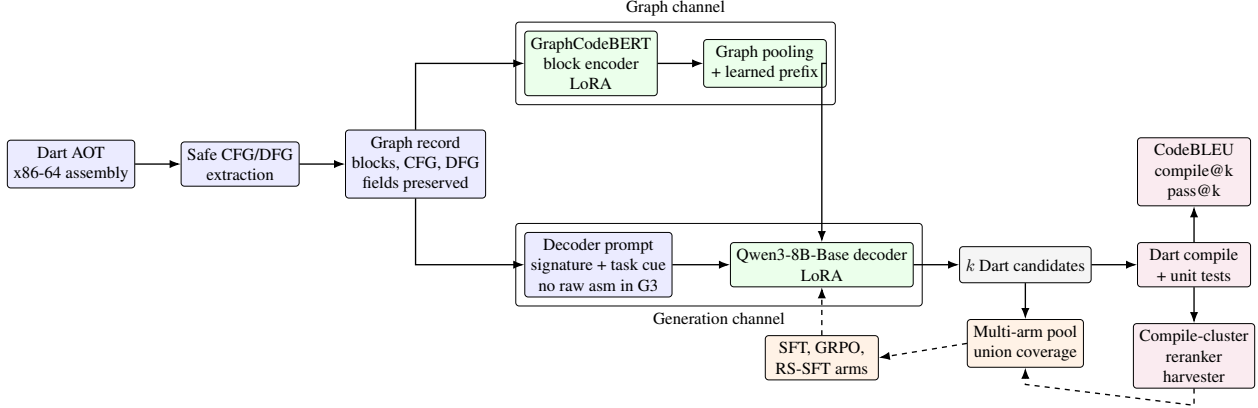


Figure 1: Overview of the graph-conditioned Dart AOT decompilation pipeline. The graph path extracts CFG/DFG structure and bounded per-block instructions from assembly and encodes them with a LoRA-adapted GraphCodeBERT encoder. Learned prefix tokens condition a LoRA-adapted Qwen3-8B-Base decoder; RMS matching is evaluated separately as a stability mechanism for wider prefix variants. Generated Dart candidates are evaluated by compile and test harnesses; the same candidate pools support GRPO rewards, rejection-sampling SFT, reranking, and multi-arm union analysis.

encoder consumes bounded per-block assembly instructions together with CFG/DFG relationships, compresses them into learned prefix tokens, and leaves only the function-level task cue in the causal decoder prompt.

3.1 Task and data

The benchmark consists of 154 filtered, test-equipped HumanEval-Dart tasks with function signatures, reference Dart implementations, AOT-derived x86-64 assembly, CFG/DFG-derived graph features, and unit tests. compile@k is evaluated on a 126-task compile/CodeBLEU set, and pass@k is evaluated on the 154-task HumanEval-Dart pass set. These denominators are reported separately: the 126-task compile/CodeBLEU set is the legacy stable subset retained for comparability with prior evaluator runs, while the 154-task pass set is the full test-equipped HumanEval-Dart split used for functional correctness and pass-set reranking.

3.2 Graph extraction

The graph pipeline extracts basic blocks from assembly using jump targets and fall-through structure, control-flow edges between blocks, lightweight data-flow edges tracking register/value relationships where available, and per-block instruction summaries with bounded instruction counts. The graph records are merged into the original JSONL rows without dropping fields such as `dart_source`, `tests`, `dart_function_signature`, and `assembly`. This is important because earlier conversion code could create graph-only records and accidentally destroy the SFT/GRPO fields required by training and evaluation.

3.3 Model architecture

The best clean architecture uses GraphCodeBERT as the graph/local block encoder, Qwen3-8B-Base as the causal decoder, learned graph prefix tokens injected into the decoder context, and LoRA on both encoder and decoder with rank 64 and alpha 128. Prefix RMS matching is reported as a stability mechanism for the wide-prefix ablation, not as an assumed property of the archived 16-prefix G3 row.

The primary graph-conditioned configuration is given below. Each entry is a single runner parameter written as `key = value in snake_case`, matching the released code:

Parameter	Value
<code>qwen_prefix_tokens</code>	16
<code>qwen_prefix_rms_match</code>	0 (for the reported 16-prefix G3 row)
<code>prompt_assembly_mode</code>	"none"
<code>prompt_fit_assembly</code>	1 (inert when <code>prompt_assembly_mode = "none"</code>)
<code>dfg_mode</code>	"edges"
<code>position_scheme</code>	"roberta"
<code>auto_cfg</code>	0
<code>max_block_instrs</code>	24
<code>graph_max_dataflow_edges</code>	4096

The `prompt_assembly_mode = "none"` setting is deliberate: the model receives structure through the graph channel rather than duplicating long assembly text in the decoder prompt. The RMS-matched p128r arm instead uses `qwen_prefix_rms_match = 1`.

3.4 Adaptation arms

We evaluate null and text-only controls, graph-text and CFG-only variants, the clean graph-only G3 architecture, wider prefix variants, GRPO reward tuning with binary/full-pass reward variants, RS-SFT from all passing candidates harvested across arms, and ultralite repair arms from GRPO, RS-SFT, and style-SFT checkpoints. Our aim is not to identify a universally dominant checkpoint, but to characterize how each intervention shifts the tradeoff among CodeBLEU, compilability, and functional correctness.

3.5 Reranking and ensembling

We distinguish intra-model reranking from multi-arm candidate ensembling. Intra-model reranking generates k candidates from one checkpoint and selects one candidate with deployable heuristics. Multi-arm candidate ensembling pools candidates from multiple independently adapted checkpoints, then measures union coverage or reranks over the pool. The deployable reranker, `compile_cluster_vote`, uses Dart compilation, structural heuristics, and candidate clustering. It does not use hidden tests. Oracle reranking uses pass/fail outcomes only to estimate the upper bound available in a sampled candidate pool.

4 Experimental Setup

All main experiments use the exact Qwen/Qwen3-8B-Base checkpoint as decoder and GraphCodeBERT as encoder. The *base reference* row is the untuned Qwen3-8B-Base decoder given the cleaned assembly prompt under the same evaluator and sampling protocol, with the learned graph-prefix channel disabled. Unless otherwise stated, evaluation uses 5 samples for `compile@k/CodeBLEU`, 10 samples for `pass@k`, maximum generation length 768 tokens, decoder prompt budget 2048 tokens, LoRA rank 64, alpha 128, and 16 graph prefix tokens. RMS matching is enabled only for runs whose labels or archived configurations explicitly include it, such as p128r.

For a task with n generated candidates and c passing candidates, `pass@k` follows the HumanEval estimator $1 - \binom{n-c}{k} / \binom{n}{k}$, averaged over tasks [5]. We use the same convention for `compile@k` by replacing test pass with successful Dart compilation. When $n = k$, as in `compile@5` with five samples and `pass@10`

with ten samples, the estimator reduces to task coverage: whether at least one candidate succeeds. $\text{pass}@1$ reduces to c/n for each task before averaging.

Multi-sample evaluation uses stochastic decoding with temperature 0.7 and nucleus sampling $p = 0.95$; single-sample evaluation is greedy. We report CodeBLEU on the compile/CodeBLEU set, compiled-only CodeBLEU, $\text{compile}@5$, $\text{pass}@1$, $\text{pass}@5$, $\text{pass}@10$, task-level union coverage across arms, and deployable/oracle reranking results.

Frontier baseline. To calibrate all specialized results against current general-purpose capability, we additionally prompt four frontier models zero-shot on the same 154 tasks: GPT-5.5 (Azure deployment alias `gpt-chat-latest`), Claude Sonnet 5, DeepSeek V4 Pro, and GLM-5.2, accessed through hosted APIs. Each model receives the full x86-64 AOT disassembly and the target function signature, with no graph channel, no fine-tuning, and no truncation to the 2048-token decoder budget used for the specialized models; the prompt is frozen after light calibration on five held-out tasks. This comparison is intentionally not architecture-controlled; it measures the capability available to a practitioner using a hosted frontier model today. Architecture-controlled comparisons within the 8B local regime are reported separately. The main comparison requests temperature 0.7, $p = 0.95$, ten samples, and a 1,024-token output cap; the archived metadata records these accepted settings for the OpenRouter models, including Sonnet 5, and for the original Azure GPT-5.5 run. Generations are scored by the identical Dart pass harness. The 154-task $\text{compile}@k$ rows use a JIT/pass-harness compile definition: we run the same candidate-plus-tests file through `dart run`, count front-end syntax/type errors as compile failures, and count runtime assertion failures or timeouts as compiled-but-not-passing. This makes $\text{compile}@k$ and $\text{pass}@k$ nested on the same candidate pools; strict AOT compilation is retained only as an additional diagnostic because it rejects some programs accepted by the JIT pass harness.

GPT-5.5 prompt ablation. We separately query the Azure GPT-5.5 deployment with seven prompt representations, ten candidates per task, an 8,192-token output cap, and deployment-default sampling shared by every ablation arm. The first serializer (*v1*) supports full assembly, a character-estimated cap of approximately 2,048 assembly tokens, full assembly plus a compact CFG topology, and compact topology without block instructions. The cap applies to assembly text rather than to the total prompt; 46/154 records show lower prompt-token accounting than their full-assembly counterpart. The revised serializer (*v2*) removes GDB dump headers and uses the cleaned instruction stream in three matched conditions: cleaned assembly only, cleaned assembly plus an address-aligned CFG with final block instructions, and all normalized instructions organized inside their CFG blocks without a separate flat listing. The *v2* conditions share the same cleaner, system prompt, Azure deployment, output cap, and deployment-default sampling, so their paired comparisons are controlled. Direct *v1*–*v2* comparisons remain descriptive because the serializer revision also changes assembly cleaning and parts of the system prompt.

We report two compile views. The legacy 126-task standalone compile metric is retained for comparability with earlier runs and CodeBLEU evaluation. The 154-task JIT/pass-harness compile metric is aligned with $\text{pass}@k$ and is used for frontier comparisons. Functional conclusions are based on $\text{pass}@k$.

5 Results

5.1 Standalone model results

The archived experiment-family labels shown in `monospace` parentheses map to the released artifact: `G3` is the clean graph-conditioned architecture, `p128r` its 128-prefix RMS-matched variant, `style1036` the

Table 1: Standalone x86 8B results. Descriptive arm names are used throughout; the archived experiment-family label is given in monospace parentheses for traceability to the released artifact. *Comp. tasks* is task coverage on the 126-task compile/CodeBLEU set: the number of tasks with at least one compilable candidate, not the total number of compilable candidates. *pass@k* is measured on 154 HumanEval-Dart tasks. No single arm is best across the functional and diagnostic metrics.

Arm	CodeBLEU	Comp. tasks	compile@5	pass@1	pass@5	pass@10
Binary-reward GRPO (binary GRPO)	0.5438	33	0.2619	0.0409	0.1280	0.1818
Rejection-sampling SFT, all arms (RS-SFT all-arms)	0.5740	54	0.4286	0.0474	0.1296	0.1688
Rejection-sampling SFT, reference (binary RS-SFT ref)	0.5323	44	0.3492	0.0357	0.1121	0.1558
Graph-conditioned (16 prefix) (clean G3)	0.6383	84	0.6667	0.0266	0.0996	0.1558
Style-SFT repair (style repair ultralite)	0.5299	25	0.1984	0.0240	0.0918	0.1429
Graph-conditioned, 128 prefix + RMS (G3 p128r)	0.6349	84	0.6667	0.0292	0.0878	0.1364
Style-SFT (1,036 ex.) (style1036 SFT)	0.5709	55	0.4365	0.0325	0.0972	0.1364
CFG-only conditioning (CFG-only)	0.6063	72	0.5714	0.0201	0.0671	0.1104
Pass@k-oriented GRPO (SimKO-style GRPO)	0.6360	84	0.6667	0.0221	0.0716	0.1104
Serialized graph-in-prompt (graph-text)	0.6253	74	0.5873	0.0227	0.0691	0.0974
Untuned Qwen3-8B-Base (base reference)	0.5383	26	0.2063	0.0156	0.0563	0.0909
Null-graph control (null graph)	0.5459	52	0.4127	0.0136	0.0494	0.0779
Text-only control (text-only)	0.5193	41	0.3254	0.0052	0.0260	0.0519
Graph-conditioned, 128 prefix, no RMS (p128 without RMS)	0.4481	5	0.0397	0.0000	0.0000	0.0000

style-matched synthetic SFT arm, and *ref*, *lite*, and *ultralite* the harvested repair-set variants used for quick consolidation checks. Appendix A gives the full label-to-configuration mapping.

Among the specialized arms, the clean G3 graph-only model has the strongest CodeBLEU and the highest legacy compile@5 task coverage on the 126-task compile set, together with competitive pass@10. The binary GRPO arm gives the highest specialized pass@10. On the legacy standalone-compile metric, GRPO compiles far less often than G3 (compile@5 0.262 versus 0.667); under the aligned 154-task JIT/pass-harness compile of Table 2, the gap is much smaller (compile@5 0.403 versus 0.422). Thus the apparent “instability” of GRPO is specific to the legacy standalone-compile measurement and does not replicate under the pass-aligned compile definition. No specialized arm is uniformly best once CodeBLEU is set aside as a diagnostic: G3 leads on similarity, GRPO leads on functional pass@k, and their compile behavior is metric-dependent.

5.2 Frontier baseline comparison

Table 2 places the specialized arms against four frontier models prompted zero-shot on the same assembly. The gap is large and one-directional. GPT-5.5 reaches pass@10 of 0.701, roughly four times the best specialized arm (0.182) and more than twice the 17-arm union coverage (0.279). Claude Sonnet 5 (0.487), DeepSeek V4 Pro (0.351), and GLM-5.2 (0.286) all exceed the union. On aligned JIT/pass-harness compile the frontier rows remain strong: GPT-5.5 reaches compile@10 of 0.955, while clean G3 and binary GRPO reach 0.630 and 0.591 respectively. None of the frontier models received any decompilation-specific training, graph conditioning, or Dart AOT adaptation. For HumanEval-scale Dart AOT functions, then, general frontier capability currently dominates the specialized 8B pipeline on functional correctness, and the specialized interventions studied here should be read as characterizing the small-local-model regime rather than as competitive with hosted frontier inference.

5.3 GPT-5.5 context and graph-serialization ablations

Table 3 tests whether the frontier gap can be attributed primarily to GPT-5.5 seeing a longer assembly prompt, and whether a textual CFG helps the frontier model. This is a separate Azure run from the canonical

Table 2: Frontier models (zero-shot, full assembly, no graph channel) versus specialized arms on the 154-task pass set. compile@k here is the JIT/pass-harness compile metric, not the legacy 126-task standalone compile of Table 1; the same extracted candidate-plus-tests file is used for compile@k and pass@k, so pass@k is nested under compile@k. pass@k remains the primary functional metric. Best in each column is in **bold**.

Model / arm	compile@5	compile@10	pass@1	pass@5	pass@10
<i>Frontier models, zero-shot</i>					
GPT-5.5	0.9495	0.9545	0.5526	0.6709	0.7013
Claude Sonnet 5	0.6365	0.6818	0.3058	0.4358	0.4870
DeepSeek V4 Pro	0.5164	0.6753	0.0851	0.2561	0.3506
GLM-5.2	0.3660	0.4286	0.1461	0.2526	0.2857
<i>Specialized arms (this work), 154-task JIT/pass-harness compile</i>					
Binary-reward GRPO	0.4033	0.5909	0.0409	0.1280	0.1818
Graph-conditioned (clean G3)	0.4215	0.6299	0.0266	0.0996	0.1558
All-arm union (17 arms)	—	—	—	—	0.2792
Untuned Qwen3-8B-Base	0.2207	0.3701	0.0156	0.0563	0.0909

Table 3: GPT-5.5 prompt-representation ablation on the same 154 tasks, with ten candidates per task. *Covered* is the pass@10 numerator. Within-version comparisons are controlled; direct v1–v2 comparisons remain descriptive because the cleaner and system prompt changed.

Prompt condition	Pipeline	pass@1	pass@5	pass@10	Covered
Full assembly	v1	0.5851	0.6894	0.7143	110
Approx. 2,048-token assembly cap	v1	0.5721	0.6718	0.7013	108
Assembly + compact CFG topology	v1	0.5864	0.6910	0.7273	112
Compact CFG topology only	v1	0.3935	0.4800	0.5065	78
Cleaned assembly only	v2	0.5792	0.6789	0.7078	109
Clean assembly + address-aligned CFG	v2	0.5695	0.6729	0.7013	108
CFG-organized block instructions, no flat listing	v2	0.5656	0.6774	0.7013	108

GPT-5.5 row in Table 2: it uses an 8,192-token output cap and deployment-default sampling, so its full-assembly row is the within-ablation control rather than a replacement for the main frontier estimate.

Within v1, the approximate assembly cap covers 108 tasks versus 110 for full assembly, with three tasks gained and five lost (exact two-sided McNemar $p = 0.727$). The observed frontier advantage therefore is not explained primarily by this context-budget difference. Adding the compact CFG topology raises the point estimate to 112 tasks, but the paired difference is also non-significant ($+3/-1$, $p = 0.625$); the data do not establish that textual CFG augmentation improves GPT-5.5. Removing block contents and providing only the compact topology reduces coverage to 78 tasks ($+5/-37$, $p = 4.43 \times 10^{-7}$). This last result shows that the particular topology-only textual summary is lossy; it does not imply that raw assembly should be fed directly to a smaller decoder.

Within v2, cleaned assembly alone covers 109 tasks. Adding the address-aligned CFG covers 108, with two tasks gained and three lost relative to cleaned assembly (exact two-sided McNemar $p = 1.000$); organizing all instructions inside CFG blocks also covers 108 ($+1/-2$, $p = 1.000$). The corresponding aligned compile@10 values are 0.9870, 0.9675, and 0.9805. Thus neither v2 textual graph representation

Table 4: Task-level uncertainty and paired tests for selected comparisons. compile@5 here is the legacy 126-task standalone-compile metric of Table 1; the aligned 154-task JIT/pass-harness compile of Table 2 does not reproduce the GRPO compile regression (see text). Coverage intervals are Wilson 95% confidence intervals. Paired tests are exact two-sided McNemar tests against clean G3; $+g/-l$ reports tasks gained/lost relative to clean G3 on the same benchmark rows. The all-arm union is pass-set coverage only.

Arm	compile@5 (95% CI)	McNemar	pass@10 (95% CI)	McNemar
clean G3 graph-only	0.6667 [0.5805, 0.7430]	–	0.1558 [0.1070, 0.2214]	–
binary GRPO	0.2619 [0.1930, 0.3449]	$+7/-58, p = 4.3 \times 10^{-11}$	0.1818 [0.1289, 0.2502]	$+7/-3, p = 0.344$
RS-SFT all-arms	0.4286 [0.3455, 0.5158]	$+11/-41, p = 3.6 \times 10^{-5}$	0.1688 [0.1179, 0.2359]	$+7/-5, p = 0.774$
CFG-only	0.5714 [0.4842, 0.6545]	$+19/-31, p = 0.119$	0.1104 [0.0701, 0.1697]	$+5/-12, p = 0.143$
text-only	0.3254 [0.2498, 0.4113]	$+9/-52, p = 1.8 \times 10^{-8}$	0.0519 [0.0266, 0.0992]	$+2/-18, p = 4.0 \times 10^{-4}$
p128 without RMS	0.0397 [0.0171, 0.0895]	$+4/-83, p = 3.0 \times 10^{-20}$	0.0000 [0.0000, 0.0243]	$+0/-24, p = 1.2 \times 10^{-7}$
all-arm union	–	–	0.2792 [0.2144, 0.3548]	$+19/-0, p = 3.8 \times 10^{-6}$

improves GPT-5.5 over the matched cleaned-assembly control, although CFG organization preserves nearly all functional and compile coverage while eliminating the separate flat listing.

That distinction is central to the local architecture. G3 is called graph-only because raw assembly is absent from the Qwen decoder prompt, not because instructions are discarded. Its GraphCodeBERT/local-block path consumes bounded block instructions and graph relationships before compressing them into 16 learned prefix tokens. Conversely, the v2 *CFG-organized block instructions* condition still contains every normalized instruction, merely grouped by block instead of repeated as a flat listing. The controlled v2 result therefore argues against claiming that textual CFG organization helps a frontier model, but it does not test the learned graph-prefix compression used by G3 or imply that raw assembly is suitable for a much smaller decoder.

Table 4 shows why the point estimates should not be overread. The binary GRPO arm has the largest standalone pass@10 numerator, but its paired gain over clean G3 is small and not statistically distinguishable at this sample size. In contrast, its compile@5 regression on the legacy 126-task standalone-compile metric is large and highly significant. We flag the scope of this compile result explicitly: it holds for the legacy standalone-compile metric only, and does not reproduce under the aligned 154-task JIT/pass-harness compile of Table 2, where GRPO and clean G3 have similar compile@5 (0.403 versus 0.422). The paired tests below should therefore be read as evidence about the legacy compile metric; the aligned JIT/pass-harness reanalysis is included in the artifact summary files and removes the pass@k > compile@k anomaly. Applying Holm–Bonferroni correction across the family of all $m = 11$ paired tests in Table 4 (five compile@5 and six pass@10 comparisons) preserves the strong effects: binary GRPO’s legacy-compile regression remains significant ($p_{\text{Holm}} = 4.3 \times 10^{-10}$), RS-SFT’s legacy-compile regression remains significant ($p_{\text{Holm}} = 2.2 \times 10^{-4}$), p128’s collapse remains significant on compile and pass ($p_{\text{Holm}} = 3.3 \times 10^{-19}$ and 9.5×10^{-7}), text-only’s pass@10 regression remains significant ($p_{\text{Holm}} = 2.0 \times 10^{-3}$), and the all-arm union remains a significant coverage gain ($p_{\text{Holm}} = 2.7 \times 10^{-5}$). The binary GRPO pass@10 gain remains non-significant after correction ($p_{\text{Holm}} = 0.688$), as do RS-SFT’s pass@10 difference ($p_{\text{Holm}} = 0.774$) and CFG-only’s compile@5/pass@10 differences ($p_{\text{Holm}} \approx 0.476$ for both). The all-arm union result is different: it adds 19 tasks over clean G3 without losing any clean-G3-covered task under the current 24-task clean-G3 count, giving a significant paired coverage gain even on the small 154-task pass set. This McNemar test is valid as a statement about paired task coverage: the union solves a strict superset of the tasks clean G3 solves. It is *not* a fair deployment comparison, because the union draws on a much larger candidate budget (17 arms $\times$ 10 samples) than the single 10-sample clean-G3 arm; we return to this budget asymmetry in Section 5.6.

5.4 Architecture ablations

The graph-conditioned arms outperform text-only and null controls on pass@10 and compile@k. The clean G3 model reaches pass@10 of 0.1558, compared with 0.0519 for the text-only G1 arm and 0.0779 for the null graph. This comparison is about representation, not about topology replacing semantics. G1 sends the full cleaned assembly listing through the causal decoder prompt with no graph prefix, whereas G3 removes raw assembly from that prompt but still encodes bounded instructions inside each basic block together with graph edges. The threefold pass@10 difference is consistent with the small decoder choking on a long, low-level assembly prompt and benefiting from learned structure-aware compression into 16 prefix tokens; it does not establish that CFG topology alone would suffice.

The current ablations do not prove that DFG edges help on their own. CFG-only outperforms graph-text on pass@10 (0.1104 versus 0.0974), and these rows do not isolate DFG under the same graph-only regime as G3. We therefore interpret the result as evidence for graph conditioning overall, while treating DFG contribution as unresolved and requiring a future controlled ablation.

The p128 experiment shows that simply increasing prefix capacity is unsafe. Without RMS matching, the 128-prefix variant collapses to zero pass@k and very low compile@5. With RMS matching, p128r recovers compile@5 and CodeBLEU but still underperforms the 16-prefix G3 model on pass@10. The lesson is that graph-prefix scale and conditioning stability matter more than raw prefix width.

5.5 GRPO and RS-SFT

GRPO improves standalone pass@10 to 0.1818, the highest of any specialized arm. On the legacy 126-task standalone-compile metric it compiles less often (compile@5 0.2619), but under the aligned 154-task JIT/pass-harness compile it is close to clean G3 (0.403 versus 0.422), so the compile cost of GRPO is metric-dependent rather than a robust regression. RS-SFT all-arms improves pass@1 and pass@5 relative to clean G3 but lowers CodeBLEU. Attempts to repair GRPO with RS-SFT recover some legacy-compile behavior but wash out the useful pass@10 tail.

This supports a careful interpretation: correctness rewards can move probability mass toward passing solutions, but the setting is small and sparse. GRPO optimizes expected per-sample reward, whereas pass@k rewards the existence of diverse correct candidates in the tail. The objectives are related but not identical. We no longer characterize GRPO as simply “unstable”: on functional metrics it is the strongest specialized arm, and its earlier apparent compile fragility is an artifact of the legacy standalone-compile metric.

5.6 Multi-arm candidate ensembling

The largest task-level coverage gain is observed when candidates are pooled across independently adapted arms, although this result reflects candidate-set complementarity rather than single-model generalization. Across 17 prediction files, 43/154 tasks have at least one passing candidate, 111/154 tasks remain zero-pass, 587 passing candidates are found before deduplication, 142 RS-SFT rows remain after dedupe and per-task capping, and 253 rows are available in the all-plus-reference RS-SFT set. The multi-arm pool combines the 14 primary model families in Table 1 with three auxiliary repair/rerank pools; the exact auxiliary labels are retained here for traceability: `withB ref lite`, `withB ref ultralite`, and `binary repair ultralite`.

This is substantially higher than the clean G3 single-arm coverage of 24 tasks and the best GRPO single-arm coverage of 28 tasks. These are candidate-coverage counts (tasks with at least one passing sample); the corresponding pass@10 estimates in Table 1 are 0.156 and 0.182. The comparison is not sample-budget matched: the union pools many more candidates than a single 10-sample arm. We therefore interpret it as a coverage ceiling and diversity diagnostic, not as evidence that a 17-arm system is the deployable default. The multi-arm result shows that the models fail differently. Even weak standalone arms can add

Table 5: Deployable compile-cluster reranking on recent H100 arms. Rates are measured on the 154-task pass-set candidate pools, not the 126-task compile/CodeBLEU set. Orig. denotes the first/original candidate, Sel. denotes the compile-cluster-selected candidate, and Oracle pass denotes a hidden-test upper-bound diagnostic.

Arm	Orig. comp.	Sel. comp.	Orig. pass	Sel. pass	Oracle pass
Reference repair, small (<code>withB ref lite</code>)	0.0584	0.4351	0.0130	0.0325	0.0519
Reference repair, smallest (<code>withB ref ultralite</code>)	0.0909	0.4351	0.0130	0.0584	0.0779
Binary-reward repair (<code>binary repair ultralite</code>)	0.0649	0.4026	0.0000	0.0844	0.1104
Style-SFT repair (<code>style repair ultralite</code>)	0.0779	0.3442	0.0260	0.0844	0.1364

new tasks to the union. The H100 ultralite label denotes a run group executed on H100 hardware, not a distinct algorithm; those arms added three new tasks to the union: `List<int> unique(List<int> list)`, `bool correctBracketing(String brackets)`, and `bool primeLength(String string)`. This complementarity motivates treating candidate ensembling as a first-class inference-time method rather than as an afterthought.

5.7 Reranking

The archived labels in Table 5 (`withB` denoting B-sweep harvest/repair candidate pools, `lite/ultralite` the smaller repair sets) are retained in parentheses so the table can be traced back to the released result files; see Appendix A.

Deployable compile-cluster reranking improves selected compile rates substantially, as shown in Table 5. The deployable reranker does not reach the oracle ceiling, but it materially improves selection. The oracle gap quantifies how much additional benefit would require better semantic selection or public tests. More importantly, for several weak arms, the reranker recovers enough compile behavior to make their candidate pools useful for downstream harvesting and analysis.

The large compile jump should be read as a reranking effect, not as a new model compile@1 probability: the reranker chooses among an existing sampled pool, so selected compile can be much higher than the original first-candidate compile rate.

6 Discussion

6.1 What graph conditioning buys

Graph conditioning improves the similarity profile and moves the model beyond text-only decoding: G3 has the strongest CodeBLEU among the specialized arms and competitive functional scores. Its benefit should be read as learned compression of local instruction content and graph relationships, not as evidence that bare topology is sufficient. This matters because the text-only 8B arm performs poorly when the causal decoder must consume the full assembly directly, while G3 encodes bounded block contents outside that prompt. We are careful not to overstate the result. Once CodeBLEU is treated as a diagnostic, G3 does not dominate the reward-tuned arms on functional metrics, and its earlier “best balanced” status rested partly on the legacy standalone-compile metric. The defensible claim is narrower than in an earlier draft: within the small-local-model regime, graph conditioning is a reasonable default that trades a little functional pass@k for markedly better similarity and a compact decoder context, but it is not the single best specialized configuration on functional grounds, and it is far from frontier capability.

6.2 Frontier models versus specialized small models

The frontier comparison is the result with the largest practical consequence. A general model with no decompilation-specific training recovers functionally correct Dart several times more often than any specialized arm, and even mid-tier frontier models exceed the full multi-arm union. The GPT-5.5 prompt ablation narrows the explanation: reducing assembly to an approximate 2,048-token budget changes coverage from 110 to 108 tasks and is not significant in the paired test. The gap therefore cannot be attributed primarily to frontier access to the full listing. Parameter and pretraining scale, broad exposure to Dart and assembly-like text, and stronger in-context semantic reconstruction remain more plausible explanations. Similarly, the compact textual CFG condition has the highest point estimate in the ablation but no significant paired advantage, so we do not claim that simply appending serialized topology improves a frontier model. The honest reading is that, for HumanEval-scale Dart AOT functions, specialized 8B training does not currently buy accuracy over a frontier zero-shot call.

This does not make specialized models pointless, but it relocates their value. The decompilation targets that motivate this work – proprietary Flutter binaries, malware, regulated or air-gapped code – are frequently exactly the inputs that cannot be sent to a hosted frontier API. In that regime the relevant comparison is not frontier-versus-specialized but specialized-versus-nothing, and a local model that runs offline at controlled cost has a genuine niche. The frontier numbers also point to the most promising way to raise the specialized ceiling: distillation, using frontier models as teachers to generate large, correct Dart AOT training corpora for small deployable students, rather than reward-tuning a small model against sparse signals. We therefore frame the specialized contributions here as characterizing and partially narrowing a gap that distillation, not on-policy reward optimization, is best placed to close.

6.3 Why reward tuning is not a shortcut

The GRPO arm has the highest specialized pass@10, and under aligned JIT/pass-harness compile it no longer looks fragile. But it does not close the frontier gap, and its gains over graph conditioning are not statistically separable at this sample size. The likely dynamics are familiar: in a group-normalized sparse-reward setting, a rare full-pass response receives a large positive advantage while most responses receive little useful semantic signal, so probability mass concentrates around a few discovered patterns. This improves candidate discovery for some tasks without producing a broadly stronger decompiler. The practical role of GRPO in this setting is as a candidate-discovery arm feeding the union, not as a standalone deployable checkpoint and not as a substitute for stronger base models or distilled data.

6.4 Why ensembling matters

pass@k is a coverage metric. If different adaptation strategies solve different tasks, then a multi-arm ensemble can reveal a larger recoverable set than any single model. The union result, 43/154, demonstrates that several apparently weak arms are still scientifically valuable because they contribute unique passing candidates.

6.5 Reranking as a bridge between research and deployment

Oracle reranking measures the sampled pool ceiling; deployable reranking tries to approximate it using available signals. Our compile-cluster reranker strongly improves selected compile rates but only partially improves selected pass@1. This suggests a clear next step: semantic rerankers that do not require hidden tests, such as learned failure predictors, signature/type consistency models, lightweight symbolic checks, or public-example execution when available.

7 Artifact and Reproducibility Notes

This draft is backed by concrete, checkable materials rather than a description of what an artifact should contain. The working repository includes the full training and evaluation code (graph extraction, GraphCodeBERT/Qwen3-8B-Base training, GRPO, rejection-sampling SFT, and reranking), the JSONL data manifests, and the generated prediction files, sweep summaries, and per-candidate metric CSVs for every arm in Table 1 and Table 4, together with the zero-shot frontier prediction files, prompts, and exact model identifiers and query dates behind Table 2. The six GPT-5.5 ablation pools in Table 3 retain provider/model metadata, prompt mode, token accounting, and, for v2, exact prompt hashes and serializer-version metadata; their aggregate and paired results are consolidated in `GPT55Abl/gpt55_ablation_summary.json`. A consolidated, deduplicated log of the exact command-line invocations used to produce these results is maintained alongside the code. A pinned local verification environment (Python 3.12.7; torch 2.11.0+cu128, transformers 5.9.0, peft 0.18.0, torch-geometric 2.7.0, accelerate 1.6.0) runs an offline regression suite – 56 preprocessing checks and 9 GRPO gradient-equivalence self-checks – before any code change is shipped to a training pod.

Two gaps remain, and we state them explicitly rather than let the materials above imply completeness. First, checkpoint weights and their SHA-256 hashes are not yet part of the archived materials: training ran on ephemeral rented GPU pods (H100/H200/B200), and only prediction outputs, not the multi-gigabyte LoRA weights, were routinely retrieved to control storage and transfer cost; the pod-side training environment, as opposed to the local verification environment above, is likewise not yet captured as a pinned manifest. Second, the run-command log is complete for every arm in Table 1 and for the union/harvesting pipeline, but the four smaller repair-set arms used only for the reranking analysis in Table 5 were run interactively on a pod, and their literal invocations are being reconstructed from pod-side logs rather than already archived.

We commit to releasing the complete artifact – code, data manifests, the full run-command log, both environment manifests, checkpoint SHA-256 hashes, and, hosting permitting, the LoRA adapter weights themselves – at an anonymized repository URL upon acceptance, consistent with double-blind review; the link will be inserted at that time, and self-identifying repository metadata and prior-work citations will be scrubbed per the venue’s anonymity policy.

The reported runs are exploratory rather than seed-averaged. We therefore treat most cross-arm comparisons as task-level evidence from fixed sampled pools rather than as estimates over random training seeds. The generation settings are fixed in the evaluation harness as described above, and GRPO rollouts use the same default temperature/top-p unless a sweep explicitly overrides them. Representative GRPO sweeps use grouped rollouts, Dart compile/test rewards, LoRA rank 64 with alpha 128, and maximum rollout lengths between 512 and 768 tokens depending on the arm. The confidence intervals and legacy compile McNemar tests in Table 4 are recomputed from archived candidate-level CSV files; pass@10 paired tests use the official aggregate pass@10 task counts, with the corrected paired-count artifact stored under `results/pass154_same_pools/`. Future artifact releases should add repeated-seed training runs.

8 Threats to Validity

Dataset size remains small: 154 pass@k tasks are enough to expose failure patterns but not enough to claim deployment readiness. The confidence intervals in Table 4 are task-level intervals over one archived generation pool, not seed-level uncertainty over retraining. Several training and evaluation arms share data, so harvest and ensemble results should be treated as optimization/coverage analysis rather than clean generalization. Some synthetic data is style-matched, but synthetic distributions may not reflect production Flutter binaries. The graph extractor is lightweight and may miss important data-flow relationships. Finally, oracle reranking uses test outcomes and is not deployable unless those tests are public; it is reported only as an

upper bound on candidate-pool quality.

Four caveats attach specifically to the frontier comparison. First, the frontier models are proprietary and versioned, and were queried at a fixed date through hosted APIs; their behavior may change, so the frontier rows should be read as a snapshot of current capability rather than a stable benchmark, and exact model identifiers and query dates are recorded in the artifact. Second, the aligned JIT/pass-harness compile@k in Table 2 is intentionally different from strict Dart AOT compilation: it answers whether the candidate is accepted by the same `dart run` front end used for pass@k, whereas strict AOT compilation can reject otherwise runnable programs. We therefore keep strict AOT compile as a diagnostic artifact rather than mixing it with pass@k in the main comparison. Third, the primary frontier comparison remains a practitioner baseline rather than an architecture-controlled comparison: model scale, training data, serving stack, and output budget all differ from the local 8B system. The GPT-5.5 cap experiment addresses only one component and uses a character-estimated assembly cap rather than an exact 2,048-token total prompt; only 46 records show reduced prompt-token accounting. Fourth, comparisons within v1 and within v2 are controlled, but direct v1–v2 comparisons are not: v2 changes disassembly cleaning and system-prompt wording in addition to graph serialization. These limitations do not affect the within-version paired tests.

9 Dual-Use Considerations

Neural decompilation can support legitimate reverse engineering, malware analysis, vulnerability research, and software maintenance, but it can also lower the cost of analyzing proprietary binaries. The experiments here use benchmark tasks rather than production applications, and the evaluation focuses on function-level recovery. Any release should avoid distributing proprietary binaries or private application code and should document that the tool is intended for authorized analysis, research reproducibility, and defensive security workflows.

10 Future Work

The frontier comparison reshapes the priorities for future work. The most promising direction is distillation: using frontier models as teachers to generate large, correct, optimization-diverse Dart/Flutter AOT corpora, then training small deployable students on that data rather than reward-tuning them against sparse signals. Two existing artifacts make this a concrete next step rather than a purely speculative agenda. First, the released clean synthetic pool contains 1,726 source/assembly pairs with executable unit tests; it can support leakage-safe held-out distillation, rejection-sampling SFT, and reward experiments at substantially larger scale than the 154-task benchmark. Second, the stage-one ARM64 Flutter artifact contains 1,714 test-equipped release-build function slices with extracted CFGs. It establishes an ARM64 data and graph-extraction path, but its controlled semantics do not yet constitute evidence on production application logic. The next cross-ISA study should create fixed train/held-out splits, repeat the graph-versus-text ablation on ARM64, and test whether the learned compression transfers beyond x86-64. Complementary directions are larger and more realistic Dart/Flutter AOT datasets; optimization-matched Swift and Dart cross-lingual experiments; learned semantic rerankers that approximate oracle selection without hidden tests; retrieval- or tool-augmented small models that narrow the in-context gap with frontier systems; graph improvements, including richer DFG extraction, block summarization, and alignment between assembly blocks and source-level constructs; multi-arm ensemble evaluation at k=25 and k=50, including frontier models in the pool, to estimate the true candidate-generation ceiling; and clean held-out train/test splits for RS-SFT and harvest experiments, separating optimization experiments from generalization claims. The prompt ablation should be extended with an exact total-token cap and a signature-only control to quantify how much HumanEval task identity can be recovered from the function signature alone. A smaller but necessary item is preserving

both compile views in future releases: aligned JIT compile for pass@k nesting, and strict AOT compile for deployment diagnostics.

11 Conclusion

This study evaluates graph-conditioned decoding, reward adaptation, rejection-sampling fine-tuning, reranking, and multi-arm candidate ensembling for Dart AOT neural decompilation, and situates all of them against frontier models prompted zero-shot on the same assembly. The dominant empirical result is the size and direction of the frontier gap: a leading general-purpose model recovers functionally correct Dart several times more often than any specialized 8B arm, and mid-tier frontier models exceed a 17-arm specialized union, despite receiving no decompilation-specific training. An approximate same-budget GPT-5.5 ablation shows that this gap is not primarily a consequence of full-assembly context. Matched v2 comparisons further show that neither adding an address-aligned textual CFG nor reorganizing all instructions by CFG significantly improves GPT-5.5 over cleaned assembly alone. For HumanEval-scale Dart AOT functions, general frontier capability currently outperforms specialized small-model training on every functional axis. Within the specialized regime that matters when binaries cannot leave a trusted environment, graph conditioning is a reasonable default because it compresses block instruction and graph information outside the causal prompt, reward tuning gives the best specialized functional scores without closing the gap, and the clearest signal is complementarity: multiple imperfect arms collectively cover substantially more tasks than any single checkpoint. The practical implications are that specialized Dart AOT decompilers should be justified by deployability, cost, and privacy rather than by out-recovering frontier models; that distillation from frontier teachers is the most promising route to a stronger deployable student; and that decompilers should be evaluated not only as single models but as candidate-generation systems paired with deployable selection mechanisms. The graph-conditioned architecture, executable synthetic pool, and ARM64 extraction artifact should therefore be read as a tested foundation for scaling and cross-ISA work, not as an endpoint or a claim of frontier parity.

A Archived Label Glossary

For full traceability to the released artifact, Table 6 maps each archived experiment-family label used in the results tables to its descriptive name and defining configuration. These labels are the literal directory/run identifiers in the artifact; they carry no meaning beyond provenance.

Table 6: Archived label to descriptive-name and configuration mapping.

Archived label	Descriptive name	Defining configuration
G3	Graph-conditioned (16 prefix)	16 graph prefix tokens, rms_match = 0
p128r	Graph-conditioned, 128 prefix + RMS	128 graph prefix tokens, rms_match = 1
p128 without RMS	Graph-conditioned, 128 prefix, no RMS	128 graph prefix tokens, rms_match = 0
CFG-only	CFG-only conditioning	control-flow edges only, no DFG
graph-text	Serialized graph-in-prompt	structure serialized into decoder prompt
null graph	Null-graph control	graph channel present but uninformative
text-only	Text-only control	decoder prompt only, no graph channel
base reference	Untuned Qwen3-8B-Base	exact Qwen/Qwen3-8B-Base, cleaned-assembly prompt, graph prefix disabled
binary GRPO	Binary-reward GRPO	GRPO with binary pass/fail reward
SimKO-style GRPO	Pass@k-oriented GRPO	GRPO with pass@k-oriented grouping
RS-SFT all-arms	Rejection-sampling SFT, all arms	SFT on passing candidates pooled across arms
binary RS-SFT ref	Rejection-sampling SFT, reference	RS-SFT restricted to the reference pool
style1036 SFT	Style-SFT (1,036 ex.)	SFT on 1,036 style-matched synthetic examples
style repair ultralite	Style-SFT repair	style-SFT repair-set consolidation run
withB ref lite/ultralite	Reference repair (small/smallest)	B-sweep reference repair pools
binary repair ultralite	Binary-reward repair	B-sweep binary-reward repair pool

References

- [1] Raafat Abualazm and Ayman AboElHassan. Llms as idiomatic decompilers: Recovering high-level code from x86-64 assembly for dart, 2026. Accepted at SANER 2026 ERA Track.
- [2] Raafat Abualazm, Ayman AboElHassan, and Amr G. Wassal. Evaluating fine-tuning and metrics for neural decompilation of dart aot binaries, 2026. arXiv:2607.06125; under review at ACM Transactions on Software Engineering and Methodology (TOSEM).
- [3] Jordi Armengol-Estape, Jackson Woodruff, Chris Cummins, and Michael F. P. O’Boyle. SLDe: A portable small language model decompiler for optimized assembly. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 67–80. IEEE, 2024.
- [4] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q. Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [6] Qibin Chen, Jeremy Lacomis, Edward J. Schwartz, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. Augmenting decompiler output with learned variable names and types. In *31st USENIX Security Symposium*. USENIX Association, 2022.
- [7] Luke Dramko, Claire Le Goues, and Edward J. Schwartz. Idioms: Neural decompilation with joint code and type definition prediction, 2025.
- [8] Nicolas Falliere. Dart AOT snapshot helper plugin to better analyze Flutter-based apps. PNF Software, JEB in Action blog. <https://www.pnfsoftware.com/blog/dart-aot-snapshot-helper-plugin-to-better-analyze-flutter-based-apps/>, 2022. JEB Decompiler Dart AOT snapshot helper plugin.
- [9] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations (ICLR)*, 2021.
- [10] Iman Hosseini and Brendan Dolan-Gavitt. Beyond the c: Retargetable decompilation using neural machine translation, 2022. Workshop on Binary Analysis Research (BAR).

- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [12] Impact-I (PT SWARM). reFlutter: Flutter reverse engineering framework. <https://github.com/Impact-I/reFlutter>, 2021.
- [13] Nan Jiang, Chengxiao Wang, Kevin Liu, Xiangzhe Xu, Lin Tan, Xiangyu Zhang, and Petr Babkin. Nova: Generative language models for assembly code with hierarchical attention and contrastive learning. In *International Conference on Learning Representations (ICLR)*, 2025.
- [14] Deborah S. Katz, Jason Ruchti, and Eric Schulte. Using recurrent neural networks for decompilation. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2018.
- [15] Jeremy Lacomis, Pengcheng Yin, Edward J. Schwartz, Miltiadis Allamanis, Claire Le Goues, Graham Neubig, and Bogdan Vasilescu. DIRE: A neural approach to decompiled identifier naming. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019.
- [16] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Remi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Mankowitz, Esme Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- [17] Peipei Liu, Jian Sun, Rongkang Sun, Li Chen, Zhaoteng Yan, Peizheng Zhang, Dapeng Sun, Dawei Wang, Xiaoling Zhang, and Dan Li. The CodeInverter suite: Control-flow and data-mapping augmented binary decompilation with LLMs, 2025.
- [18] Yu Meng, Mengzhou Xia, and Danqi Chen. SimPO: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [19] Ruotian Peng, Yi Ren, Zhouliang Yu, Weiyang Liu, and Yandong Wen. SimKO: Simple pass@k policy optimization, 2025.
- [20] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. CodeBLEU: a method for automatic evaluation of code synthesis, 2020.
- [21] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [22] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3473–3487. Association for Computational Linguistics, 2024.
- [23] Hanzhuo Tan, Xiaolong Tian, Hanrui Qi, Jiaming Liu, Zuchen Gao, Siyi Wang, Qi Luo, Jing Li, and Yuqun Zhang. Decompile-bench: Million-scale binary-source function pairs for real-world binary decompilation, 2025.

- [24] Worawit Wangwarunyoo. B(l)utter: Flutter mobile application reverse engineering tool. <https://github.com/worawit/blutter>, 2023.
- [25] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. ReSym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2024.
- [26] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Wang, Zhenru Cui, Zhenru Zhang, and Zihan Qiu. Qwen3 technical report, 2025.
- [27] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025.